

Kilocal — Documentazione API pubbliche

Versione: 1.0

Data: 1 giugno 2026

Progetto: kilocal-shop-fe (+ estensione CMS `shop-apis`)

1. Panoramica

Il frontend shop (Nuxt) espone due livelli di API accessibili dal browser:

Livello	Base URL	Descrizione
API Shop	<code>{SHOP_URL}/api/*</code>	Endpoint custom per carrello, ordini, utenti, indirizzi, PayPal
Proxy CMS	<code>{SHOP_URL}/cms/*</code>	Proxy verso Directus (auth, GraphQL, REST, asset)
Route di utilità	<code>{SHOP_URL}/finder,</code> <code>{SHOP_URL}/logout</code>	Routing interno e logout

URL di esempio (produzione):

- Shop: `https://shop.kilocalprogram.it`
- CMS: `https://cms.kilocalprogram.it`

Formato risposta: JSON (`Content-Type: application/json`), salvo redirect su `/finder` e `/logout`.

CORS: consentito solo dagli origin configurati (`SHOP_URL`, `MAIN_URL`, `PUBLIC_URL`, PayPal).

2. Autenticazione

2.1 Sessione utente (cookie)

Cookie	Descrizione
<code>klkl-data</code>	Token di sessione Directus (access + refresh)
<code>klkl_refresh_token</code>	Refresh token

- Durata: 7 giorni
- `SameSite: lax`, `Secure` in produzione
- Le richieste autenticate devono usare `credentials: 'include'`

2.2 Login

Il login non passa da `/api/*`. Avviene tramite Directus SDK sul proxy CMS:

```
POST {SHOP_URL}/cms/auth/login
```

```
Content-Type: application/json
```

```
{  
  
  "email": "utente@example.com",  
  
  "password": "*****"  
}
```

Dopo il login, le chiamate autenticate inviano automaticamente `Authorization: Bearer {access_token}` (gestito dal client Directus).

2.3 Logout

```
GET {SHOP_URL}/logout
```

Invalida i cookie di sessione e reindirizza a `/?logout=true`.

2.4 Livelli di accesso

Livello	Endpoint
Pubblico	Carrello (init, items, discount), settings, registrazione, password reset

Autenticato	Ordini, indirizzi utente, PayPal
Solo server	Chiamate interne al CMS con <code>CMS_TOKEN</code> (non esposte al client)

3. API Shop (`/api/*`)

3.1 Settings

Restituisce configurazione globale del shop (menu, stringhe, footer).

GET `/api/settings`

Autenticazione: no

Risposta 200:

```
{  
  
  "settings": {  
  
    "main_menu": { "...": "..." },  
  
    "footer_menu": { "...": "..." },  
  
    "generic_strings": { "...": "..." }  
  
  }  
}
```

In caso di errore restituisce `{ "settings": {} }`.

3.2 Carrello

3.2.1 Inizializza / recupera carrello

POST `/api/cart/init`

Autenticazione: opzionale (se loggato, recupera l'ultimo carrello dell'utente)

Body:

```
{  
  
  "storeCartUUID": "uuid-opzionale-del-carrello-salvato"  
}
```

Logica:

1. Se `storeCartUUID` è presente, cerca un carrello `draft/pending` con quell'UUID
2. Se l'utente è loggato, cerca il suo ultimo carrello attivo
3. Se il carrello anonimo appartiene a un altro utente → errore

4. Se l'utente ha un carrello diverso da quello salvato → il carrello salvato viene annullato
5. Altrimenti crea un nuovo carrello

Risposta 200: oggetto `Carts` completo (uuid, items, totali, shipping, discounts, ecc.)

Errori:

Codice	Messaggio
—	<code>Mismatch between stored cart and user.</code>

3.2.2 Aggiungi / aggiorna item

POST `/api/cart/{uuid}/items/upsert`

Autenticazione: no

Body:

```
{
  "entry": {
    "collection": "products",
    "item": "123"
  },
  "quantity": 1,
  "fixQta": false,
  "variant": {
    "variantId": "456",
    "title": "Variante M",
    "barcode": "8001234567890",
    "assetId": "file-uuid"
  },
  "kitVariants": null
}
```

Campo	Tipo	Obbligatorio	Note
<code>entry.collection</code>	string	sì	"products" o "product_kits"
<code>entry.item</code>	string	sì	ID prodotto/kit

<code>quantity</code>	number	sì	Quantità desiderata
<code>fixQty</code>	boolean	sì	<code>false</code> = somma alla qty esistente; <code>true</code> = imposta qty assoluta
<code>variant</code>	object null	no	Solo per prodotti con varianti
<code>kitVariants</code>	object null	no	Scelte varianti per kit (<code>Record<productId, KitVariantChoice></code>)

Risposta 200: carrello aggiornato (dopo refresh totals)

Errori:

Codice	Messaggio
—	<code>Cart not found</code>

3.2.3 Rimuovi item

DELETE /api/cart/{uuid}/items/delete

Body:

```
{
  "item": "cart_item_id"
}
```

Risposta 200: carrello aggiornato

3.2.4 Applica codice sconto

PATCH /api/cart/{uuid}/discounts/apply

Body:

```
{
  "code": "PROMO2026"
}
```

Risposta 200: carrello aggiornato

Errori:

HTTP	statusMessage	Descrizione
400	<code>Missing required parameters</code>	Codice mancante

400	DISCOUNT_ALREADY_APPLIED	Sconto già applicato
400	DISCOUNT_INVALID	Sconto non valido per il carrello
400	DISCOUNT_MAX_USES_REACHED	Limite utilizzi raggiunto
404	DISCOUNT_NOT_FOUND	Codice inesistente
404	Cart not found	UUID carrello non valido

3.2.5 Rimuovi codice sconto

DELETE /api/cart/{uuid}/discounts/delete

Body:

```
{
  "code": "PROMO2026"
}
```

Errori:

HTTP	Messaggio
400	Missing required parameters
404	Cart not found
404	Discount not found

3.2.6 Salva indirizzi sul carrello

PATCH /api/cart/{uuid}/addresses/store

Body:

```
{
  "delivery": { "id": "address-uuid" },
  "invoice": { "id": "address-uuid" }
}
```

`invoice` è opzionale; se omissso, viene usato l'indirizzo di spedizione.

Risposta 200: carrello aggiornato con shipping calcolato

3.3 Ordini

3.3.1 Crea / aggiorna ordine

POST /api/orders/upsert

Autenticazione: sì (401 se non loggato)

Body (creazione):

```
{  
  
  "total": 89.90,  
  
  "cart": "cart-uuid",  
  
  "shipping_address": "address-id",  
  
  "shipping_method": "shipping-id",  
  
  "invoice_address": "address-id"  
  
}
```

Body (aggiornamento):

```
{  
  
  "order": "order-uuid",  
  
    "cart": "cart-uuid",  
  
  "total": 89.90,  
  
  "shipping_address": "address-id",  
  
  "shipping_method": "shipping-id",  
  
  "invoice_address": "address-id",  
  
  "status": "pending",  
  
  "paypal_order": { "...": "..." }  
  
}
```

Risposta 200: array con l'ordine aggiornato/creato:

```
[  
  
  {  
  
    "uuid": "order-uuid",  
  
    "status": "draft",  
  
    "total": 89.90,  
  
    "paypal_order": null  
  
  }  
  
]
```

```
}
```

```
]
```

Errori:

HTTP	statusMessage
401	Unauthorized
400	Cart not found or missing in payload
400	MISSING_SHIPPING_ADDRESS
400	MISSING_SHIPPING_METHOD
400	Cart not found
500	ORDER_UPSERT_FAILED

3.4 PayPal

3.4.1 Crea ordine PayPal

POST /api/paypal

Autenticazione: sì

Body:

```
{  
  
  "cartId": "cart-uuid",  
  
  "orderId": "order-uuid"  
}
```

Risposta 200: risposta PayPal Checkout Orders API v2 (oggetto ordine con `id`, `status`, `links`, ecc.)

Errori:

HTTP	statusMessage
400	MISSING_CART_ID
400	MISSING_ORDER_ID
401	Sessione scaduta. Per favore effettua nuovamente il login.
404	ORDER_NOT_FOUND
404	CART_NOT_FOUND

3.5 Utenti

3.5.1 Registrazione

POST /api/users

Autenticazione: no

Body:

```
{  
  "first_name": "Mario",  
  "last_name": "Rossi",  
  "email": "mario@example.com",  
  "password": "*****",  
  "password_confirm": "*****",  
  "privacy": true,  
  "newsletter": false,  
  "is_platform_user": false  
}
```

Risposta 200: oggetto utente Directus creato (ruolo `User`, `registered_from: shop`)

Errori:

HTTP	statusMessage
400	Email is required
400	Passwords do not match

3.5.2 Password dimenticata

POST /api/auth/password-forgotten

Body:

```
{  
  "email": "mario@example.com"  
}
```

Risposta 200: { "status": "ok" } (sempre, anche se l'email non esiste — per privacy)

Errori:

HTTP	statusMessage
400	EMAIL_REQUIRED

Genera un token di reset e avvia il flusso email via Directus.

3.5.3 Reset password

POST /api/auth/password-reset

Body:

```
{
  "token": "reset-token-from-email",
  "password": "nuovaPassword",
  "password_confirm": "nuovaPassword"
}
```

Risposta 200: { "status": "ok" }

Errori:

HTTP	statusMessage
400	TOKEN_REQUIRED
400	PASSWORDS_DO_NOT_MATCH
400	NO_USER_FOUND_BY_RESET_TOKEN

3.6 Indirizzi utente

Tutti richiedono autenticazione. L'utente può operare solo sui propri indirizzi.

3.6.1 Crea indirizzo

POST /api/user-addresses

Body:

```
{
  "first_name": "Mario",
  "last_name": "Rossi",
  "company": "",
  "country": "it",
}
```

```
"address": "Via Roma 1",  
"city": "Milano",  
"province": "MI",  
"zip": "20100",  
"phone": "+39 333 1234567",  
"is_default": false  
}
```

Risposta 200: { "data": { ...indirizzo } }

3.6.2 Aggiorna indirizzo

PATCH /api/user-addresses/{id}

Body: stessi campi della creazione (parziali ammessi)

Errori: 401 Unauthorized, 403 Forbidden (indirizzo di altro utente)

3.6.3 Elimina indirizzo

DELETE /api/user-addresses/{id}

Risposta 200: { "data": { "id": "..." } }

4. Route di utilità

4.1 Finder (risoluzione URL pagine CMS)

GET /finder?p={internal_name}&l={locale}¶ms[slug]={value}

Parametro	Descrizione
p	Nome interno pagina CMS (es. <code>shop_single_product</code> , <code>shop-account</code>)
l	Locale (es. <code>it</code> , <code>it-IT</code>) — default <code>it</code>
params[key]	Parametri da interpolare nello slug (es. <code>params[slug]=nome-prodotto</code>)

Comportamento:

- Con `Accept: application/json` → { "path": "/percorso-risolto" }

- Altrimenti → redirect HTTP 302 al path risolto
- Se `p` manca → redirect a `/`

Esempio:

```
GET /finder?p=shop_single_product&params[slug]=kit-base-uomo
```

```
→ 302 /kit-base-uomo
```

5. Proxy CMS (`/cms/*`)

Tutte le richieste a `{SHOP_URL}/cms/{path}` vengono inoltrate a `{CMS_URL}/{path}`.

Endpoint Directus più usati dal frontend

Metodo	Path	Uso
POST	<code>/cms/auth/login</code>	Login
POST	<code>/cms/auth/refresh</code>	Refresh token
POST	<code>/cms/auth/logout</code>	Logout
POST	<code>/cms/graphql</code>	Query/mutation GraphQL (contenuti, pagine, prodotti)
GET	<code>/cms/items/{collection}</code>	REST Directus (permessi pubblici/autenticati)
GET	<code>/cms/assets/{id}</code>	Asset/media

Nota: i permessi effettivi dipendono dalla configurazione ruoli Directus. Il proxy non aggiunge autenticazione server-side: usa il token utente dal cookie se presente.

6. API CMS interne (`shop-apis`)

Endpoint sul CMS, non chiamato direttamente dal browser ma dal server shop durante il refresh del carrello.

```
POST {CMS_URL}/shop-apis/{cartUUID}/refresh
```

```
Content-Type: application/json
```

```
{
```

```
  "query": "...GraphQL GetCart...",
```

```
"variables": {  
  
  "filter": { "uuid": { "_eq": "{cartUUID}" } }  
  
}
```

Funzione:

1. Ricalcola totali carrello (subtotal, shipping, discount)
2. Applica sconti automatici
3. Aggiorna il record `carts` su Directus
4. Restituisce il carrello completo via GraphQL

Chiamato internamente da tutte le operazioni carrello che restituiscono il carrello aggiornato.

7. Modello dati carrello (risposta tipica)

Dopo init/upsert/delete/apply discount, la risposta segue lo schema GraphQL `Cart`:

Campo	Tipo	Descrizione
<code>uuid</code>	string	Identificativo pubblico carrello
<code>subtotal</code>	number	Subtotale prodotti
<code>subtotal_no_discount</code>	number	Subtotale senza sconti
<code>total</code>	number	Totale finale
<code>total_no_discount</code>	number	Totale senza sconti
<code>total_items</code>	number	Numero articoli
<code>shipping_cost</code>	number	Costo spedizione (scontato)
<code>shipping_no_discount</code>	number	Costo spedizione pieno
<code>shipping</code>	object	Metodo spedizione selezionato
<code>items[]</code>	array	Righe carrello (prodotto/kit, qty, prezzi)
<code>discounts[]</code>	array	Sconti applicati
<code>discounts_raw</code>	object	Dettaglio sconti per prodotto/ordine
<code>user_created</code>	object	Utente proprietario (se loggato)

